zer0Track — Team Role Outlines

PDF conversion of docs/zer0Track_Role_Outlines.docx, content unchanged. Detailed build plan, synced file structure, and function naming — so everyone works from the same map.

Shared Repo Structure

Lock this structure before anyone starts coding. Changes to models.py or schema.md go through the group chat first.

```
zer0track/
|-- schema.md -- locked -- don't edit without a group call
|-- backend/ -- Bhone
|   |-- models.py
|   |-- checkers.py
|   |-- policy.py
|   |-- audit.py
|   `-- api.py
|-- agent/ -- Kogu
|   |-- investigator.py
|   |-- tools.py
|   `-- injection_tests.py
|-- data/ -- Raja
|   |-- generate_data.py
|   `-- dataset/ -- generated .json files, one per case
|-- evaluation/ -- Raja
|   |-- run_eval.py
|   |-- policy_variants.py
|   `-- metrics.py
|-- dashboard/ -- Nour
|   |-- app.py
|   `-- views/
|      |-- case_list.py
|      |-- timeline.py
|      `-- approval.py
`-- docs/ -- Win
    |-- zer0Track_Proposal_Final.pdf
    `-- pitch_deck.pptx
```

Bhone — Backend/policy

- Set up the API skeleton — FastAPI + Pydantic models for Shipment Event, Courier History (schema sections 1-2).
- Build the three mandatory checkers, each returning the shared shape (checker_name, passed, confidence, detail): Parcel/code integrity, Identity/authority, Location.
- Build the policy table (Section 3 of the proposal) as a lookup function: takes the three PASS/FAIL/UNKNOWN results → returns decision, recommended_action, requires_human_approval (schema section 4 shape).
- Transfer-code handling — bind code to parcel/transaction/receiver, reject on reuse, atomic consume-on-accept.
- Audit log — append-only, hash-linked, stores Approval Records (schema section 5).
- Expose endpoints Kogu and Nour can call: submit case → run checks → get decision; approve/reject case.

Files & Functions. Depends on nothing — can start immediately with schema data. Unblocks Kogu (checkers), Nour (decision shape), Win (integration).

Kogu — Agent/security

- Wire the LLM to the checkers (mocked first, real once Bhone ships them) — agent calls Bhone's checker functions, never re-runs them itself.
- Build the investigator logic — when a mandatory check is UNKNOWN/conflicting, LLM picks ONE allowlisted read-only lookup: delegation records, checkpoint logs, or custody history.
- Enforce the tool allowlist — fixed set of read-only tools, arguments validated against schemas, no free-form actions.
- Write the reasoning/explain step — plain-language trace + evidence IDs + unresolved questions (not a confidence score).
- Prompt-injection test suite — run these against the agent and document pass/fail: fake "approved by supervisor" text, an instruction to skip the location check, a forged system-style instruction in a photo caption.
- Hand Bhone the decision object in the exact Agent Decision Output shape (schema section 4) so the policy service can revalidate it.

Files & Functions. Depends on Bhone's checker output shape. Can build/test against mocks before Bhone finishes.

Raja — Data/evaluation

- Generate synthetic shipment events matching the schema exactly (Shipment Event + Courier History), labelled: normal, fraud_gps_spoof, fraud_social_engineered_otp, fraud_ghost_delivery, ambiguous.
- Target 20 labelled cases covering: normal handoffs, GPS-spoofed deliveries, socially-engineered OTP, replayed transfer codes, and valid edge cases (authorised delegation, depot collection).
- Build the comparison harness — run all 20 cases through three configurations: scan-only baseline, fixed rules only, rules+LLM.
- Build 2-3 investigation-policy variants (stricter/looser triggers) to show the detection-vs-false-alert tradeoff.
- Compute and report metrics: invalid-proof detection rate, false-alert rate, unsupported acceptances, unresolved cases, tool calls, time-to-recommendation.
- Write up results as observed — including misses, not a pre-claimed accuracy number.

Files & Functions. Depends on the schema only to start generating data; needs Bhone's and Kogu's working pipeline to actually run the comparison once built.

Nour — Frontend/dashboard/demo

- Build against mock data first (schema sections 4-5 shapes) — don't wait on Bhone/Kogu.
- Case list view — pending/flagged/escalated cases, searchable by shipment ID.
- Investigation timeline view — for one case: checks run (PASS/FAIL/UNKNOWN), tools called, evidence shown, final decision, in chronological order.
- Approval UI — supervisor can approve/reject/hold with a comment, writes an Approval Record.
- Swap mock data for real API calls once Bhone's endpoints are live.

Files & Functions. Depends on nothing to start (mocked); depends on Bhone and Kogu for the real end-to-end flow before the demo is final.

Win — Integration/proposal/demo

- Hold the 30-min schema-lock call (if not already done) so sections 1-5 of the schema are frozen before everyone builds.
- Mid-build integration checks — periodically wire Bhone's API + Kogu's agent + Nour's dashboard together end-to-end, catch shape mismatches early rather than at the last minute.
- Final end-to-end wiring before the demo — confirm the 3 demo runs actually flow through all four people's real code, not mocks.
- Assemble the written proposal — the proposal document stays the source of truth; update it as the build evolves.
- Build and rehearse the 3-run demo on top of the finished dashboard: (1) condo guardhouse scan → delegation lookup finds valid delegate → resolved; (2) same scan, no authorisation → pending, supervisor may hold; (3) replayed transfer code → blocked regardless of recommendation.
- Time the rehearsal — know exactly how long each run takes before pitch day.
- Build the pitch deck from the proposal's structure (Problem → Inputs → Workflow → Safeguards → Demo → Limitations → Team).
- Own the submission checklist — confirm what GDEX x ANON x UTAR actually requires (file formats, deadlines, page/slide limits) and that everything's in before cutoff.

Files & Functions. No new source files to own — Win's job is making sure the four folders above actually call each other correctly (dashboard/app.py → backend/api.py → backend/checkers.py + agent/investigator.py) and keeping the proposal and pitch deck in sync with what the code actually does by demo day.

One rule for everyone: don't rename a function or change a shape in models.py / schema.md without flagging it in the group chat first — that's the thing most likely to break someone else's work silently.